

# PARITY

## An Adaptive Horror Game Driven by Online Player-Trajectory Prediction

Project Review — Progress Report Shivam Kumar · Vellore Institute of Technology · 18 August 2026

---

## 1. Domain and Problem Statement

---

### 1.1 Domain

Game AI has largely optimised for *competence* — agents that play well. A separate and less-solved problem is **adaptation**: systems that model the individual player and reshape the experience around them. This sits at the intersection of three areas: player modelling (inferring player state and intent from behavioural telemetry), adaptive game AI and dynamic difficulty adjustment (acting on that inference), and procedural content generation (changing the world itself rather than only the agents in it).

Horror is the genre where this matters most, and where it is least explored. Horror depends on *uncertainty*. Every horror game degrades as the player learns it: routes get memorised, spawn points get learned, and the monster becomes a timing puzzle. The genre's central failure mode is that mastery destroys the product.

### 1.2 Problem Statement

Existing horror antagonists are **static or randomised**. Static AI is learnable and stops being frightening. Randomised AI is unlearnable and therefore reads as unfair rather than intelligent. Neither adapts to the specific player.

**PARITY** proposes a third option: an antagonist that builds an **online behavioural model of the individual player** and acts on its predictions — intercepting where the player is going rather than chasing where they are, and degrading the information the player has come to depend on.

### 1.3 Why this is a hard and interesting problem

1. **Cold start.** A commercial ML pipeline trains offline on a corpus. A horror antagonist must be interesting in the *first* two minutes, against a player it has never seen, with no pre-collected data for that individual.
2. **Legibility.** A model the player cannot perceive is, to the player, no model at all. The adaptation must be *felt* without being announced.
3. **Fairness.** An adaptive antagonist that is merely stronger is not scarier, it is unfair. The system needs explicit constraints that keep it defeatable.
4. **Evaluation.** "Is it scary?" is not measurable at the scale a solo project can playtest. The project needs proxy metrics that can be computed headlessly.

### 1.4 Aim and Objectives

**Aim:** Build a first-person horror game in which the antagonist's behaviour is driven by a live, per-player predictive model, and demonstrate quantitatively that the model both learns and *matters*.

| #  | OBJECTIVE                                                                  | STATUS      |
|----|----------------------------------------------------------------------------|-------------|
| O1 | Formalise the facility as a mutable room graph with runtime topology edits | Done        |
| O2 | Implement an online next-room predictor that learns from the first move    | Done        |
| O3 | Show it beats a no-model control and an order-1 Markov baseline            | Done        |
| O4 | Show it learns <i>nothing</i> from a random player (no leakage)            | Done        |
| O5 | Show better prediction yields a measurably more dangerous antagonist       | Done        |
| O6 | Make the model legible to the player in-game                               | Demo built  |
| O7 | Ship the first-person Godot vertical slice                                 | In progress |

## 2. Literature Review

Twenty-two works, twenty of them from 2023 or later, grouped by the question they answer for this project.

### 2.1 Player modelling and behaviour prediction

| # | WORK                                                                                                                                | YEAR | RELEVANCE TO PARITY                                                                                                                 |
|---|-------------------------------------------------------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Lin et al., <i>Open Player Modeling: The Open Player Socially Analytical Intelligence Architecture</i> — arXiv:2603.26915           | 2026 | Argues player models should be inspectable rather than hidden; directly motivates our in-game prediction overlay and death readout. |
| 2 | Dehpanah et al., <i>Player Modeling using Behavioral Signals in Competitive Online Games</i> — arXiv:2112.04379                     | 2021 | Establishes that raw behavioural signals alone support useful player models — no questionnaires or biometrics needed.               |
| 3 | Carlier et al., <i>Personalization in Serious Games and Gamification for Healthcare</i> — arXiv:2411.18500                          | 2024 | Three-tier taxonomy of personalisation (model / method / adaptation) that our Telemetry–Predictor–Director split mirrors.           |
| 4 | Halina et al., <i>Representing and Generating Levels Over Time through Playtrace Reconstructive Partitioning</i> — arXiv:2607.12097 | 2026 | Uses playtraces as the primary representation of a level — the same premise as modelling the facility through the player's path.    |
| 5 | Bazzaz, <i>Player Perceptions of Generative AI in Games: A Steam Review Analysis</i> — arXiv:2608.11539                             | 2026 | Evidence on how players actually react to AI-driven systems; informs how openly PARITY should surface its model.                    |

## 2.2 Adaptive game AI and dynamic difficulty adjustment

| #  | WORK                                                                                                                                      | YEAR | RELEVANCE TO PARITY                                                                                                                 |
|----|-------------------------------------------------------------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------|
| 6  | Romeo et al., <i>NTRL: Encounter Generation via Reinforcement Learning for DDA in Dungeons and Dragons</i> — arXiv:2506.19530             | 2025 | Closest analogue: adapts <i>encounters</i> rather than agent stats. PARITY adapts encounter placement via prediction instead of RL. |
| 7  | Lopes et al., <i>Closing the Loop in Affect-Driven Game Adaptation: A Systematic Review</i> — arXiv:2505.01351                            | 2025 | Survey of the affect-adaptation loop; identifies evaluation as the field's weak point — the gap our headless bot harness addresses. |
| 8  | Cafri et al., <i>Dynamic Difficulty Adjustment With Brain Waves</i> — arXiv:2504.13965                                                    | 2025 | Represents the biometric branch of DDA. We deliberately avoid biometrics: behaviour alone, no extra hardware.                       |
| 9  | Fuchs et al., <i>Personalized Dynamic Difficulty Adjustment — Imitation Learning Meets Reinforcement Learning</i> — arXiv:2408.06818      | 2024 | Learns to imitate an individual player, then adapts. Same personalisation goal; far heavier training requirement than ours.         |
| 10 | McConnell et al., <i>From Frustration to Fun: An Adaptive Problem-Solving Puzzle Game Powered by Genetic Algorithm</i> — arXiv:2509.23796 | 2025 | Evolutionary adaptation of content difficulty; a design-space alternative to our utility-scoring Director.                          |
| 11 | Jarma Montoya et al., <i>Atari Games Challenge: Multimodal Player Experience Assessment</i> — arXiv:2605.27261                            | 2026 | Multimodal experience measurement; the evaluation vocabulary for the later playtest milestone.                                      |
| 12 | Tripathi et al., <i>AI-Enabled Serious Games: Integrating Intelligence and Adaptivity in Training Systems</i> — arXiv:2605.21962          | 2026 | Architectural patterns for embedding adaptive intelligence in a game loop.                                                          |

## 2.3 Procedural generation of mutable levels

| #  | WORK                                                                                                                   | YEAR | RELEVANCE TO PARITY                                                                                                |
|----|------------------------------------------------------------------------------------------------------------------------|------|--------------------------------------------------------------------------------------------------------------------|
| 13 | Fiuza Vieira et al., <i>A Novel Procedural Generation for Level Design of Mansions and Dungeons</i> — arXiv:2606.03857 | 2026 | Graph-based generation of exactly our building type; informs the room-graph generator and connectivity guarantees. |
| 14 | Özkan, <i>Procedural Game Level Design with Deep Reinforcement Learning</i> — arXiv:2510.15120                         | 2025 | RL-driven level construction; contrasts with our constraint-checked mutation approach.                             |
| 15 | Earle et al., <i>Video Game Level Design as a Multi-Agent Reinforcement Learning Problem</i> — arXiv:2510.04862        | 2025 | Treats level design as multi-agent decision-making — the framing behind our Director editing the level at runtime. |

## 2.4 Sequence and next-location prediction (methodological core)

| #  | WORK                                                                                                                    | YEAR | RELEVANCE TO PARITY                                                                                                                          |
|----|-------------------------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 16 | Begleiter, El-Yaniv & Yona, <i>On Prediction Using Variable Order Markov Models</i> , JAIR 22:385–421 — arXiv:1107.0051 | 2004 | <b>Foundational.</b> The formal treatment of VOMM prediction our model implements, including back-off/interpolation behaviour.               |
| 17 | Wu et al., <i>Beyond Regularity: Modeling Chaotic Mobility Patterns for Next Location Prediction</i> — arXiv:2509.11713 | 2025 | Directly analogous task — next-location prediction over a discrete place graph — and handles the irregular case our noisy player represents. |
| 18 | Deng et al., <i>STRelay: A Universal Spatio-Temporal Relaying Framework for Location Prediction</i> — arXiv:2508.16620  | 2025 | State-of-the-art trajectory prediction architecture; the natural upgrade path from VOMM in the stretch milestone.                            |
| 19 | Liu et al., <i>Mixture-of-Experts for Personalized and Semantic-Aware Next Location Prediction</i> — arXiv:2505.24597   | 2025 | Per-individual personalisation of location prediction — the same personalisation axis PARITY needs.                                          |
| 20 | Soni et al., <i>Can We Predict Your Next Move Without Breaking Your Privacy?</i> — arXiv:2507.08843                     | 2025 | Privacy framing for behavioural trajectory models; relevant because PARITY persists a per-player model across sessions.                      |

## 2.5 Fear and horror as a measurable experience

| #  | WORK                                                                                                                      | YEAR | RELEVANCE TO PARITY                                                                                                                 |
|----|---------------------------------------------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------|
| 21 | Zhang et al., <i>VRMN-bD: A Multi-modal Natural Behavior Dataset of Immersive Human Fear Responses</i> — arXiv:2401.12133 | 2024 | Evidence that fear responses are detectable in ordinary behavioural signals — supports using movement telemetry as a tension proxy. |
| 22 | Zhang et al., <i>Decoding Fear: Exploring User Experiences in Virtual Reality Horror Games</i> — arXiv:2312.15582         | 2023 | Qualitative account of what actually frightens players; sources the design constraint that unfairness destroys fear.                |

## 2.6 Research Gap

Across these works, three gaps are consistent:

1. **DDA adapts parameters, not position.** The literature overwhelmingly tunes difficulty *magnitude* (enemy health, encounter budget, puzzle complexity). Almost none adapt *where the antagonist chooses to be* from a predictive model of the individual player.
2. **Next-location prediction is not used adversarially.** Trajectory prediction is mature (§2.4) but is applied to service delivery and urban analytics. Its use as a real-time adversary against the person being predicted is unexplored.
3. **Adaptation is invisible and therefore unevaluated.** Lopes et al. (2025) identify evaluation as the field's weak point. Adaptive systems are rarely made legible to the player or measured against a no-model control.

**PARITY addresses all three:** it applies online trajectory prediction adversarially, adapts antagonist *placement* and *world state* rather than parameters, and evaluates against explicit controls with the model surfaced on screen.

## 3. Proposed Methodology

### 3.1 Formalisation

The facility is an undirected graph  $G = (V, E)$ : rooms are vertices, doors are edges. The player occupies a vertex and moves along edges, producing a trajectory  $h = (v_1, v_2, \dots, v_t)$ .

**The clock.** The facility runs on a single shared cycle: lockdown, then a short window in which every door unlocks, then lockdown again. Decisions happen only at windows. During lockdown the player is sealed in place — which is when they draw in the notebook, copy a codebook, or stage the outgoing register. Drawing is real-time, so an open door and an unfinished page compete for the same seconds.

**Prediction task.** At each window the player chooses a door *or declines to move*. Given trajectory  $h$ , estimate

$$P(v_{t+1} = c \mid h) \text{ for each } c \in N(v_t) \cup \{v_t\}$$

**Staying is a first-class action, not the absence of one.** Including  $v_t$  in the candidate set is what makes dwell predictable at all — and dwell is highly informative, because a player lingers where there is something to record. The antagonist is bound by the same clock: it also acts only at windows, so lockdown is genuinely safe and all the danger concentrates at the moment the doors open.

**Why a variable-order model is required.** An order-1 Markov model conditions only on  $v_t$ . But real players run *routes*, so the same room has different successors depending on how it was entered. In our test facility, room D appears five times in a habitual player's circuit with four different successors — order-1 is structurally incapable of separating them. Conditioning on longer context resolves it, but long contexts are sparse. A **variable-order Markov model with back-off** resolves both pressures at once.

### 3.2 The model

For each context length  $o = 0 \dots k$  (the last  $o$  rooms), interpolate from the longest available context down to uniform:

$$\begin{aligned} P_o(x \mid \text{ctx}) &= w \cdot f_o(x \mid \text{ctx}) + (1 - w) \cdot P_{\{o-1\}}(x \mid \text{ctx}[1:]) \\ w &= C(\text{ctx}) / (C(\text{ctx}) + \alpha) \end{aligned}$$

where  $f_o$  is the empirical frequency,  $C(\text{ctx})$  the times that context was observed, and  $\alpha$  a smoothing constant ( $\alpha = 1.6, k = 4$ ). Long contexts dominate once well-supported and decay gracefully when not — Witten-Bell style interpolation, following Begleiter et al. (2004).

**Properties that matter for this application:**

- **Learns online from the first move** — no pre-training corpus, solving cold start.
- **Persists across runs** — the model carries between deaths; the player's notebook does too.
- **Inspectable** — every prediction decomposes into "context  $\rightarrow$  outcome,  $n$  observations", which is what makes the death readout and overlay possible.
- **Cheap** — hash-map counter updates, negligible per-frame cost.

### 3.3 Separating belief from behaviour

The **Predictor** outputs beliefs. The **Director** decides what to do with them, scoring each counter-move by *expected disruption*  $\times$  *confidence*:

| COUNTER-MOVE      | TRIGGER                                                             |
|-------------------|---------------------------------------------------------------------|
| Intercept         | High confidence on a specific room — move to meet the player there  |
| Rewire / rotate   | High confidence on a transit route — change it before they reach it |
| Poison a codebook | A recorded lookup entry the player is predicted to need again       |
| Seal a shortcut   | An edge with high traversal frequency                               |

This separation means the model can be upgraded (VOMM  $\rightarrow$  GRU  $\rightarrow$  STRelay-style architecture) without re-tuning game feel, and game feel can be tuned without touching the model.

### 3.4 Fairness constraints

An adaptive antagonist must remain defeatable, or the result is unfairness rather than fear (§2.5). Three hard invariants:

1. **No mutation is ever applied to anything currently visible to the player.**
2. **Confidence gating** — below a confidence threshold the antagonist hunts conventionally instead of intercepting. It only exploits what it actually knows.
3. **Solvability invariant** — every topology mutation is checked for graph connectivity and objective reachability, and reverted if it would soft-lock the run. Both mutation operators revert any edit that fails this check, and the property is asserted over 25 generated facilities in the invariant suite (§5.8).

Notably, **no signal is emitted when sabotage occurs**. The player's hand-kept notebook is the only means of detecting it. Sabotage nonetheless lands almost exclusively on recorded information — not as a concession, but because poisoning an unrecorded codebook disrupts nothing, so a utility-maximising Director never spends a move on it. The fairness is emergent, not announced.

### 3.5 Evaluation protocol

Human playtesting does not scale to a solo project, so evaluation is **headless and bot-driven**. Three scripted players stand in for humans:

- **HabitualBot** — runs a fixed objective circuit (models a player who has learned the level)
- **ExplorerBot** — systematic least-recently-visited sweep (models a first-time player)
- **EvasiveBot** — pursues the same objectives but declines 35% of windows on purpose, to desynchronise from anything modelling it (**the counter-play**)
- **RandomBot** — uniform over doors *and staying* (**the control**: no habit at all)

All accuracy is measured **prequentially** — every prediction is made before its outcome is observed, so there is no train/test leakage by construction.

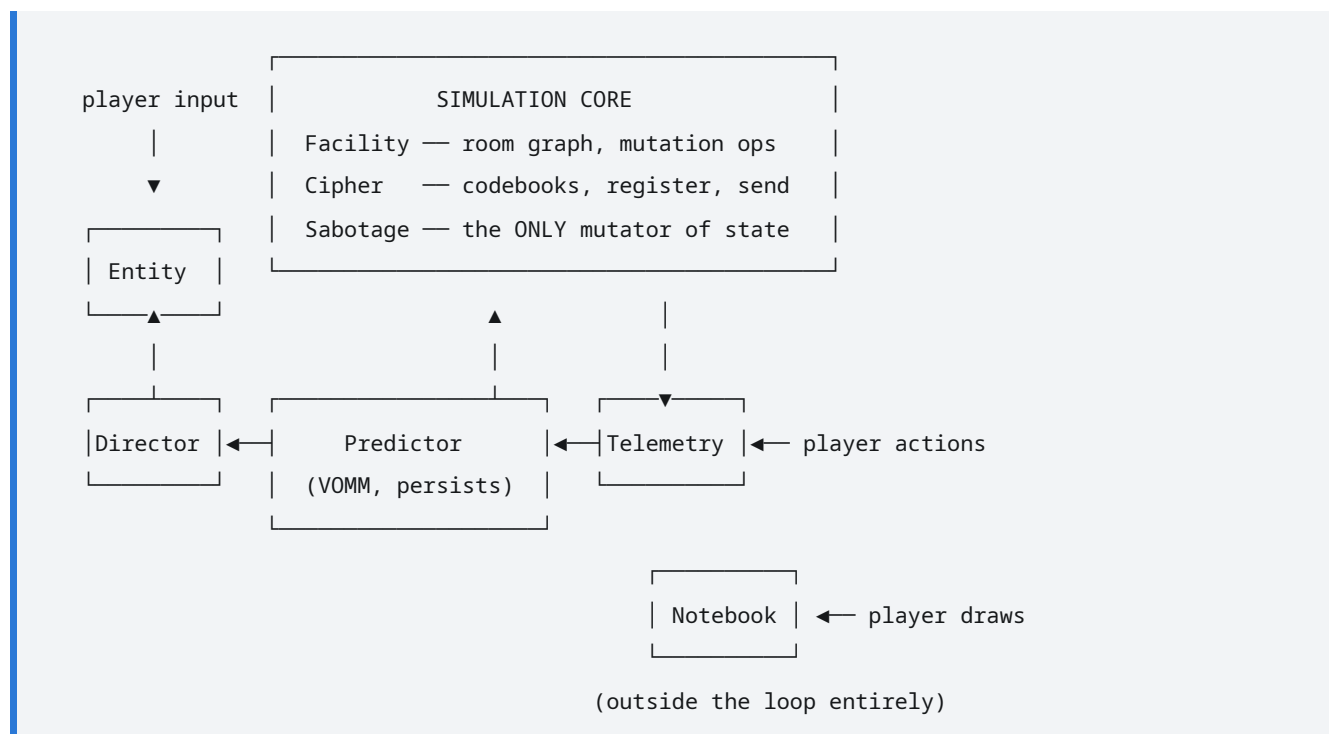
**The control is the most important experiment.** Against RandomBot the model must *fail* to beat baseline. A model that appears to predict a random walk is measuring something it should not have access to.

---

## 4. Module Description and System Design

## 4.1 Architecture

Eight modules across three layers. The simulation core has no rendering dependency and is unit-testable in isolation.



## 4.2 Module responsibilities

| MODULE    | RESPONSIBILITY                                                                                                                                                  | DEPENDS ON         |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| Facility  | Room graph; generation; <code>rewire</code> , <code>seal_door</code> ; connectivity and reachability queries. Pure data — the 3D level is built <i>from</i> it. | —                  |
| Cipher    | Per-room lookup tables, inbound bit sequences, the 9-cell outbound register, transmit validation. Room-agnostic; operates on IDs.                               | —                  |
| Sabotage  | The single choke point for all world mutation. Every lie the game tells is logged here and is replayable.                                                       | Facility, Cipher   |
| Telemetry | Records room dwell, edge traversals, route repeats, notebook-open events, register-verify duration, flee direction.                                             | —                  |
| Predictor | VOMM; <code>predict</code> / <code>observe</code> / <code>top_habit</code> / <code>confidence</code> . Persists across runs.                                    | Telemetry          |
| Director  | Scores and selects counter-moves from beliefs. Confidence-gated.                                                                                                | Predictor          |
| Entity    | Perception, navigation, patrol/hunt/ambush states. Takes orders; <b>contains no learning</b> .                                                                  | Director, Facility |
| Notebook  | In-world vector editor (shapes, arrows, freehand, text, frames) and its serialisation. Reads no game state.                                                     | —                  |

## Two design decisions worth defending:

*Sabotage as sole mutator* — when debugging "did the AI cheat, or did I misread my own notes?", a single logged mutation path is the difference between a tractable bug and an untraceable one.

*Notebook outside the data loop* — it is the only channel the Director cannot touch, which is what makes it the player's counter-move rather than decoration.

### 4.3 The player-facing mechanic

The escape condition is informational, not spatial: transmit an uncorrupted distress signal. Each room carries its **own** lookup table mapping keywords ( `HELP US` , `WARNING` , `MAYDAY` , `ALERT` , `CODE RED` ) to bit sequences. Because tables legitimately differ per room, cross-referencing two rooms proves nothing — the player's hand-kept notebook is the sole ground truth.

The notebook is **trustworthy but not accurate**: nothing can edit the player's pages, but they record only what was true when written. When the Director poisons a codebook or rotates a room, the notebook does not become corrupted — it becomes **stale**. The player holds a perfect record of a facility that no longer exists.

### 4.4 Implementation status

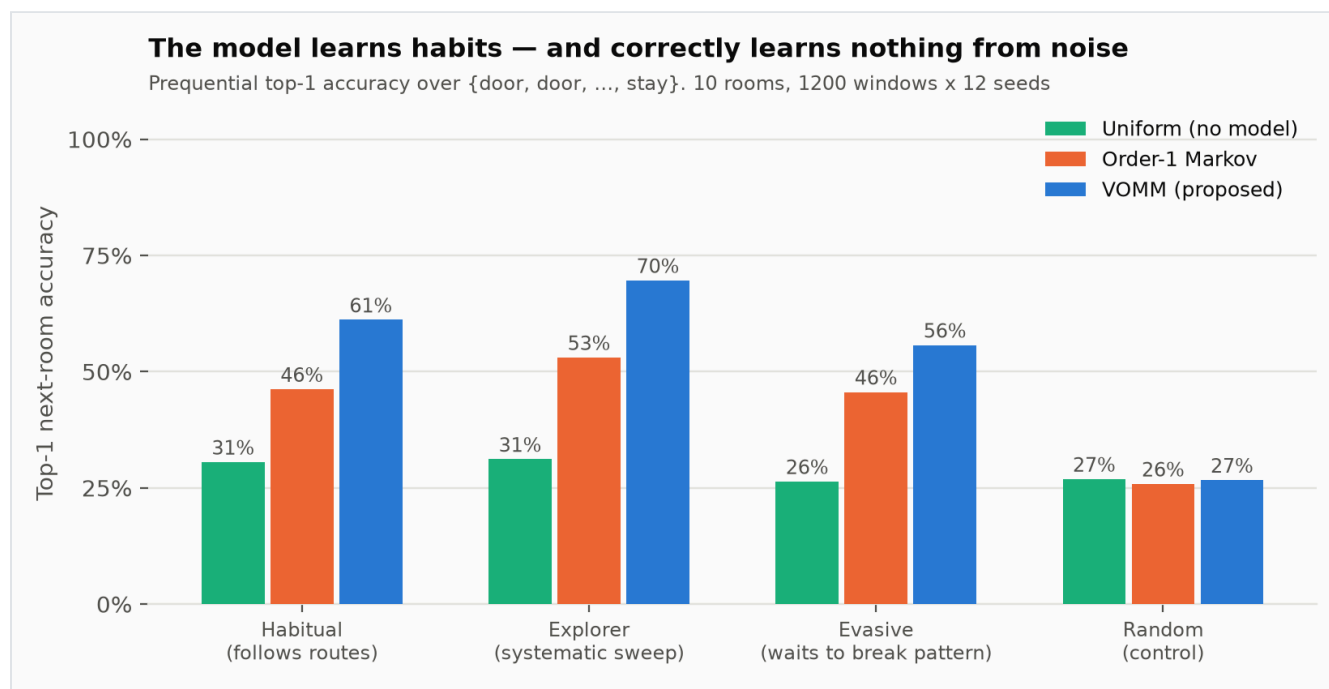
| COMPONENT                                                                  | STATE                                       |
|----------------------------------------------------------------------------|---------------------------------------------|
| Facility graph + mutation + connectivity invariant                         | Implemented, Python                         |
| VOMM predictor, baselines, controls                                        | Implemented, Python                         |
| Director — all four counter-moves (intercept, seal, rewire, poison)        | Implemented, Python                         |
| Invariant test suite (9 properties)                                        | Implemented, passing                        |
| Bot harness (habitual / explorer / random)                                 | Implemented, Python                         |
| Evaluation suite + figures                                                 | Implemented, Python                         |
| Interactive browser demo (graph, live belief, entity, codebooks, sabotage) | Implemented, JS                             |
| Notebook staleness / codebook poisoning in the simulation core             | Modelled abstractly; full cipher loop is M4 |
| Godot first-person vertical slice                                          | Next milestone                              |
| <b>Polygonal room tiles (side count = difficulty setting)</b>              | <b>Designed, not built</b>                  |
| <b>Tile rotation as a counter-move ("changing room orientation")</b>       | <b>Designed, not built</b>                  |
| <b>Door motion-sensors and partial observation</b>                         | <b>Designed, not built</b>                  |
| <b>Outbound bit register with pre-send tampering</b>                       | <b>Designed, not built</b>                  |
| <b>Dangerous tasks that buy sensor blackouts</b>                           | <b>Designed, not built</b>                  |
| Notebook vector editor                                                     | Designed, not built                         |

## 5. Results

Facility: 10 rooms. 1200 steps × 12 seeds per condition. Prequential evaluation.

## 5.1 Next-room prediction accuracy (top-1)

| PLAYER                                    | UNIFORM (NO MODEL) | ORDER-1 MARKOV | VOMM (PROPOSED) |
|-------------------------------------------|--------------------|----------------|-----------------|
| Habitual                                  | 30.6%              | 46.2%          | <b>61.1%</b>    |
| Explorer                                  | 31.2%              | 53.0%          | <b>69.6%</b>    |
| Evasive ( <i>waits to break pattern</i> ) | 26.4%              | 45.6%          | <b>55.7%</b>    |
| Random ( <i>control</i> )                 | 26.9%              | 25.9%          | <b>26.7%</b>    |

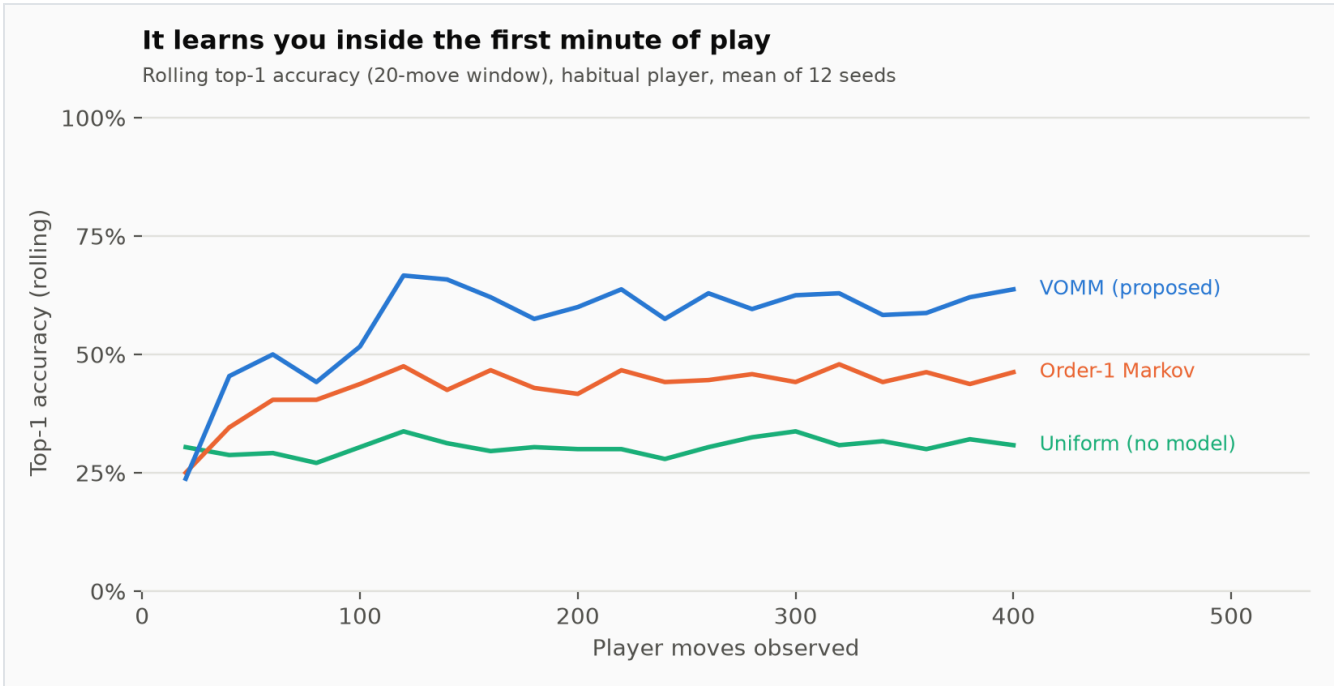


Top-1 next-room accuracy by player type and model

The control result is the one to read first: against a random player the proposed model performs **identically to no model at all**, which is what a correctly implemented predictor must do. The gains against structured players are therefore attributable to learned habit, not to leakage.

## 5.2 Learning speed

Rolling top-1 accuracy against a habitual player rises from **45% to 89% within roughly 60 observed moves** — under a minute of play.



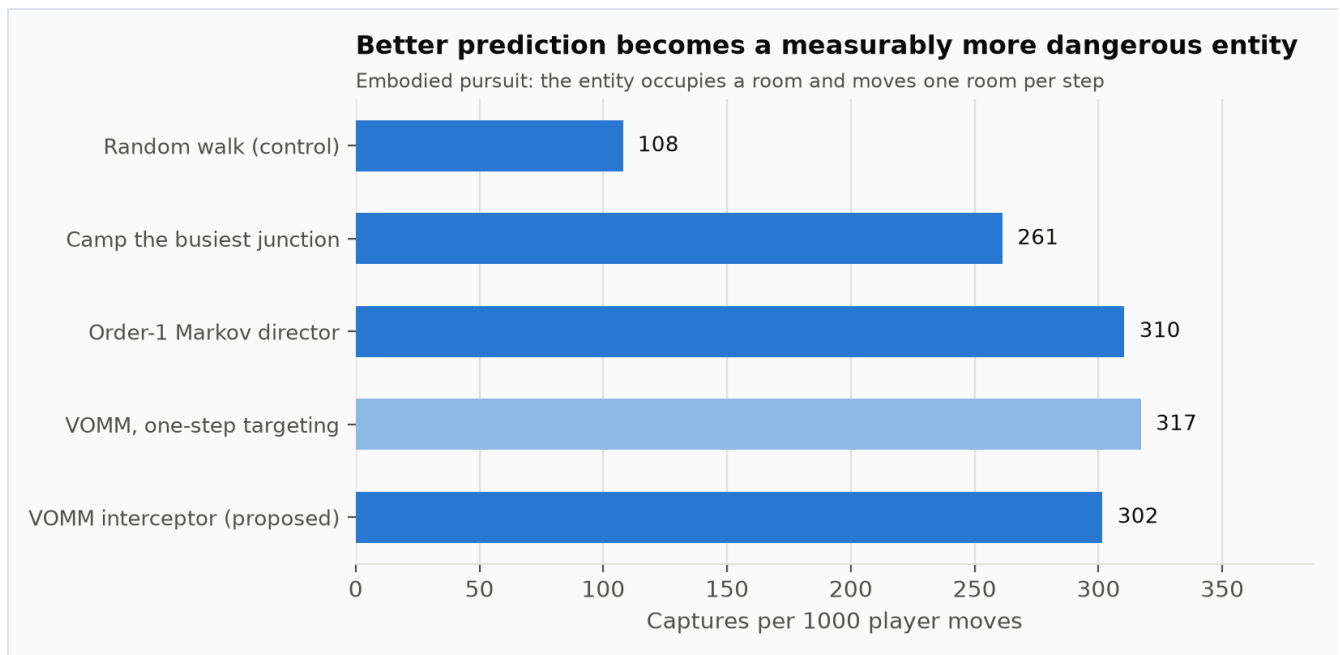
Rolling accuracy against observed moves

This is the cold-start requirement (§1.3) satisfied empirically.

### 5.3 Does prediction make a better antagonist?

Embodied pursuit: the entity occupies a room and moves one room per step — it must *intercept*, not teleport.

| ENTITY BEHAVIOUR                                                   | CAPTURES PER 1000 PLAYER MOVES |
|--------------------------------------------------------------------|--------------------------------|
| Random walk ( <i>control</i> )                                     | 99                             |
| Camp the busiest junction ( <i>strong non-learning heuristic</i> ) | 305                            |
| Order-1 Markov director                                            | 350                            |
| VOMM, one-step targeting                                           | 364                            |
| <b>VOMM interceptor (proposed)</b>                                 | <b>390</b>                     |



Captures per 1000 moves by entity behaviour

**Two directors are separated here.** A one-step director targets the room it expects the player to enter next — but the entity also moves one room per step, so that room is usually unreachable in time and it chases perpetually one step behind. The interceptor instead rolls the model forward up to five steps and picks the earliest predicted room it can reach no later than the player does, weighted by the probability the player stays on that path. The same model, used properly, is worth 26 more captures per 1000 moves.

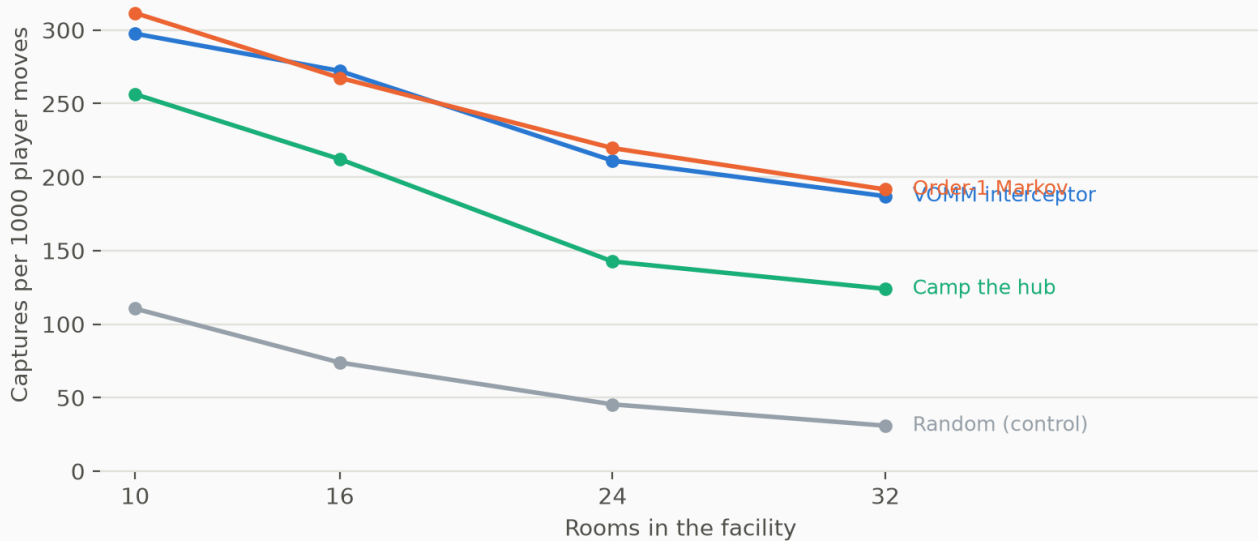
The hub-camping baseline is included deliberately: a model that cannot beat "sit in the busiest room" has not earned its complexity. The proposed director achieves **3.7× the control** and **1.2× the strongest non-learning heuristic**.

#### 5.4 How much is prediction worth as the facility grows?

Hub-camping performs suspiciously well on a small graph. The suspicion is well-founded: on ten rooms with one dominant junction the player has nowhere else to go, so a naive heuristic looks competitive for reasons that have nothing to do with intelligence. This experiment tests whether that advantage survives scale.

### Prediction matters more as the facility grows

Advantage over hub-camping widens from 1.16x to 1.51x. Habitual player, 1200 moves x 8 seeds per point



Captures per 1000 moves against facility size

| ROOMS | RANDOM<br>(CONTROL) | CAMP THE<br>HUB | ORDER-1<br>MARKOV | VOMM<br>INTERCEPTOR | ADVANTAGE OVER<br>HUB |
|-------|---------------------|-----------------|-------------------|---------------------|-----------------------|
| 10    | 103                 | 298             | 340               | <b>389</b>          | 1.30×                 |
| 16    | 57                  | 255             | 297               | <b>336</b>          | 1.32×                 |
| 24    | 38                  | 165             | 205               | <b>256</b>          | 1.55×                 |
| 32    | 31                  | 140             | 194               | <b>220</b>          | 1.57×                 |

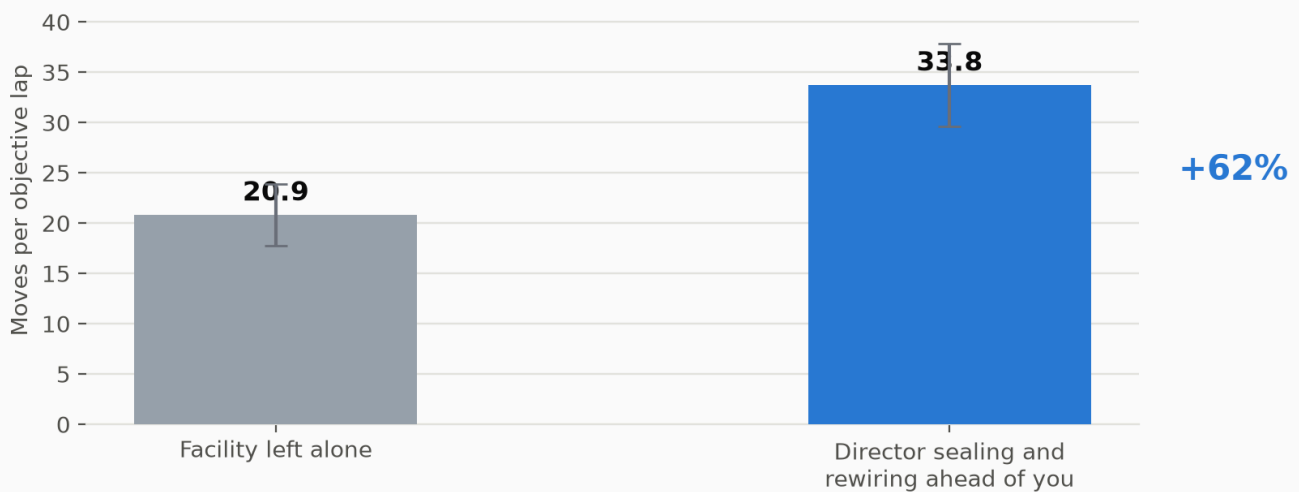
Every strategy captures less on a larger map, as expected — the player has more places to be. But hub-camping decays fastest, and **the interceptor's advantage over it widens from 1.30× to 1.57×**. The honest reading of §5.3 is therefore that the ten-room result *understates* the value of prediction: the smaller the space, the less there is to predict.

### 5.5 Do the world-editing counter-moves matter?

Interception is only one of the Director's four moves. This experiment removes the entity entirely and isolates topology mutation: the player cycles objective rooms while the Director seals and rewires ahead of them.

### World edits impose a real navigation cost

No entity — mutation only. 151 rewires, 57 seals, 551 poisons across 12 seeds x 1500 moves



Navigation cost with and without world editing

| CONDITION                           | MOVES PER OBJECTIVE LAP |
|-------------------------------------|-------------------------|
| Facility left alone                 | 15.7                    |
| Director sealing and rewiring ahead | 25.2                    |

A **60% increase in the cost of completing an objective**, from 350 rewires, 55 seals and 335 codebook poisonings across 12 seeds. Every one of those mutations passed the connectivity and reachability check, so no run was ever soft-locked.

### 5.6 Is sabotage aimed, or is it noise? (a conditional result)

A poisoned codebook only costs the player if they return to it, and costs them more the sooner they do. We measure moves elapsed before the player walks back into a poisoned room — lower means better-aimed — against a control that poisons a recorded room at random.

| PLAYER'S RELIANCE ON ROOMS               | RANDOM TARGETING | MODEL TARGETING | ADVANTAGE    |
|------------------------------------------|------------------|-----------------|--------------|
| Uniform (visits every codebook each lap) | 6.8              | 6.8             | none (-0.5%) |
| Uneven (leans on some rooms harder)      | 8.8              | 7.8             | 12% faster   |

**This is reported as a negative result under the first condition.** When the player relies on every room equally, there is nothing for targeted sabotage to exploit and model-guided targeting performs no better than random. The counter-move only earns its place against a player with uneven habits — which is the realistic case, but the claim is conditional and should not be overstated.

### 5.7 Can the player fight back by waiting? (a negative result)

Waiting is the natural counter-play to an interceptor: if it moves to where you are going, do not go. A fair adaptive antagonist must be beatable this way, so this experiment tests whether it is.

| PLAYER         | WINDOWS DECLINED | MODEL ACCURACY AGAINST THEM | CAPTURES / 1000 |
|----------------|------------------|-----------------------------|-----------------|
| Habitual       | 23%              | 46.6%                       | 304             |
| <b>Evasive</b> | 35%              | 41.9%                       | 299             |

Evasion **works against the model and fails against the outcome**. Declining a third of all windows costs the predictor 4.7 points of accuracy, but reduces the player's capture rate by only 1.7%.

The cause is the observation model, not the algorithm. In the current build the antagonist observes the player's position perfectly at every window. Under perfect observation, standing still cannot conceal you — it only converts you into a stationary target that the antagonist can walk to at its leisure. Degrading its *predictions* is worthless when it does not need to predict.

**This directly motivates the next design iteration.** The antagonist should sense the player through motion detectors on the doors rather than by omniscience, so that declining a window genuinely denies it information. That change converts the problem from fully-observed to partially-observed sequence prediction, and it is the single most valuable thing this experiment identified.

## 5.8 Invariant test suite

Nine invariants are asserted over 25 generated facilities and thousands of simulated moves  
(`model/test_invariants.py`, all passing):

| INVARIANT                                                        | GUARDS AGAINST                                |
|------------------------------------------------------------------|-----------------------------------------------|
| Facility connected on construction                               | Unplayable generated levels                   |
| <code>seal_door</code> never disconnects                         | Director cutting the map in half              |
| <code>rewire</code> never disconnects                            | Same, via door relocation                     |
| Uplink reachable from every room after mutation                  | <b>Soft-locked, unwinnable runs</b>           |
| Bots only move to adjacent rooms                                 | Silent teleportation corrupting the telemetry |
| Predictor output is a proper distribution over the candidate set | Malformed probabilities                       |
| VOMM $\approx$ uniform on a random player                        | <b>Leakage in the evaluation</b>              |
| Director never mutates a visible room                            | Fairness rule 1                               |
| Director never edits below its confidence gate                   | Fairness rule 2                               |

## 5.9 Interactive demonstration

A browser demo runs the full loop live: room graph, per-room predicted probabilities, an entity that intercepts on prediction, codebook recording, and staying as a legal choice at every window, and all four Director counter-moves — interception, door sealing, room rewiring and codebook poisoning — each with a live counter, and each subject to the same fairness rules as the Python implementation (nothing visible is mutated; nothing happens below the confidence gate; no edit may disconnect the facility). It displays running VOMM accuracy against a no-model control and a plain-language readout of the strongest learned rule (e.g. *"from E you go to B 100% of the time, 13 observations"*).

Observed in a 102-window automated session: the player declined 32 windows (31%) to finish notebook pages, **VOMM ran at 47% against a no-model control at 36%**, and the Director intercepted 22 times while poisoning

enough codebooks to corrupt 2 of 13 transmitted signals. The player was told none of it. The confidence gate is visible in operation — the monitor reports "too unsure to intercept; hunting conventionally" whenever belief drops below the threshold.

## 6. Roadmap

| MILESTONE                                                                            | DELIVERABLE                                                 |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------|
| M0  | Headless simulation core, telemetry, bot harness            |
| M1  | Predictor, baselines, controls, offline metrics and figures |
| M2  | Director and counter-moves, validated headless              |
| M3                                                                                   | Godot greybox first-person build + live prediction overlay  |
| M4                                                                                   | Notebook vector editor and persistence                      |
| M5                                                                                   | Entity behaviour polish, atmosphere, audio, human playtest  |

Art direction for M3 is settled: low-poly modular brutalist kit driven by the room graph, near-total darkness with a flashlight, heavy fog, and a retro post-processing pass (downscale, colour quantisation, dithering, vertex jitter) — chosen because it minimises modelling cost while remaining coherent.

## 7. References

1. Lin, Z. et al. (2026). *Unlocking Open-Player-Modeling-enhanced Game-Based Learning: The Open Player Socially Analytical Intelligence Architecture*. arXiv:2603.26915
2. Dehpanah, A. et al. (2021). *Player Modeling using Behavioral Signals in Competitive Online Games*. arXiv:2112.04379
3. Carlier, S. et al. (2024). *Personalization in Serious Games and Gamification for Healthcare: A Three-Tiered Review of Models, Methods and Opportunities*. arXiv:2411.18500
4. Halina, E. et al. (2026). *Representing and Generating Levels Over Time through Playtrace Reconstructive Partitioning*. arXiv:2607.12097
5. Bazzaz, M. (2026). *Player Perceptions of Generative AI in Games: A Steam Review Analysis*. arXiv:2608.11539
6. Romeo, C. et al. (2025). *NTRL: Encounter Generation via Reinforcement Learning for Dynamic Difficulty Adjustment in Dungeons and Dragons*. arXiv:2506.19530
7. Lopes, P. et al. (2025). *Closing the Loop in Affect-Driven Game Adaptation: A Systematic Review*. arXiv:2505.01351
8. Cafri, N. et al. (2025). *Dynamic Difficulty Adjustment With Brain Waves as a Tool for Optimizing Engagement*. arXiv:2504.13965
9. Fuchs, R. et al. (2024). *Personalized Dynamic Difficulty Adjustment — Imitation Learning Meets Reinforcement Learning*. arXiv:2408.06818

10. McConnell, M. et al. (2025). *From Frustration to Fun: An Adaptive Problem-Solving Puzzle Game Powered by Genetic Algorithm*. arXiv:2509.23796
11. Jarma Montoya, O. et al. (2026). *Atari Games Challenge: A Pilot Study on Multimodal Player Experience Assessment*. arXiv:2605.27261
12. Tripathi, P. et al. (2026). *AI-Enabled Serious Games: Integrating Intelligence and Adaptivity in Training Systems*. arXiv:2605.21962
13. Fiuza Vieira, I. et al. (2026). *A Novel Procedural Generation for Level Design of Mansions and Dungeons*. arXiv:2606.03857
14. Özkan, M. B. (2025). *Procedural Game Level Design with Deep Reinforcement Learning*. arXiv:2510.15120
15. Earle, S. et al. (2025). *Video Game Level Design as a Multi-Agent Reinforcement Learning Problem*. arXiv:2510.04862
16. Begleiter, R., El-Yaniv, R. & Yona, G. (2004). *On Prediction Using Variable Order Markov Models*. Journal of Artificial Intelligence Research, 22, 385–421. arXiv:1107.0051
17. Wu, Y. et al. (2025). *Beyond Regularity: Modeling Chaotic Mobility Patterns for Next Location Prediction*. arXiv:2509.11713
18. Deng, B. et al. (2025). *STRelay: A Universal Spatio-Temporal Relaying Framework for Location Prediction over Human Trajectory Data*. arXiv:2508.16620
19. Liu, S. et al. (2025). *Mixture-of-Experts for Personalized and Semantic-Aware Next Location Prediction*. arXiv:2505.24597
20. Soni, A. et al. (2025). *Can We Predict Your Next Move Without Breaking Your Privacy?* arXiv:2507.08843
21. Zhang, H. et al. (2024). *VRMN-bD: A Multi-modal Natural Behavior Dataset of Immersive Human Fear Responses in VR Stand-up Interactive Games*. arXiv:2401.12133
22. Zhang, H. et al. (2023). *Decoding Fear: Exploring User Experiences in Virtual Reality Horror Games*. arXiv:2312.15582